

StayAI Localization

Technische Probleme und umgesetzte Lösungen

Ziel der Anpassungen

Das Script korrigiert fehlende deutsche UI-Texte sowie Datum-, Uhrzeit- und Währungsformate direkt im DOM. Der Fokus lag darauf, dynamische React-Updates nicht zu blockieren, bestehende Node-Strukturen zu erhalten und Formatierungen mehrfach sicher ausführen zu können.

1. Währungsformatierung

Mehrfach formatierte Währungswerte

PROBLEM

Währungswerte wurden bei mehreren Script-Runs erneut verarbeitet. Dadurch konnten fehlerhafte Werte wie 89,92,00 EUR entstehen.

LÖSUNG

Die Währungsformatierung wurde idempotent umgesetzt. Bereits korrekt formatierte deutsche EUR-Beträge werden erkannt und nicht erneut umgeschrieben.

Preisbestandteile in getrennten Text-Nodes

PROBLEM

StayAI rendert Preise teilweise als getrennte Text-Nodes, zum Beispiel ein Node für EUR beziehungsweise € und ein weiterer Node für 269.76. Eine reine Text-Node-Regel konnte deshalb keinen vollständigen Betrag erkennen.

LÖSUNG

Es wurde eine Spezialbehandlung für gesplittete Currency Nodes ergänzt. Sie erkennt den gemeinsamen Parent-Node, identifiziert Symbol- und Amount-Node und formatiert den Betrag gezielt im Amount-Node.

React-kontrollierte Amount-Nodes dürfen nicht ersetzt werden

PROBLEM

Eine Zwischenlösung hat Symbol und Betrag zu einem einzelnen formatierten String zusammengeführt. Dadurch konnte StayAI beziehungsweise React den ursprünglichen Amount-Node bei späteren Preisupdates nicht mehr sauber aktualisieren.

LÖSUNG

Die ursprüngliche Text-Node-Struktur bleibt erhalten. Nur das separate Währungssymbol wird geleert, während der Amount-Node formatiert wird. So bleibt der dynamische Betrag weiterhin React-kontrolliert.

Alte und neue Preise wurden kombiniert

PROBLEM

Beim Wechsel von 4 auf 8 Wochen blieb der alte formatierte Preis teilweise im DOM stehen und wurde mit dem neuen Preis kombiniert, zum Beispiel 89,92 EUR269.76.

LÖSUNG

Der Parent-Node wird mit `data-stayai-split-currency="true"` markiert. Bei späteren `characterData-Updates` wird gezielt der aktualisierte Amount-Node erneut erkannt und formatiert.

2. Seitenwechsel und dynamische Inhalte

Kurz sichtbare englische Texte nach Navigation

PROBLEM

Bei SPA-Routenwechseln wurde die neue Seite erst von StayAI gerendert. Dadurch waren englische Texte kurz sichtbar, bevor das Script erneut lief.

LÖSUNG

Routenwechsel werden erkannt und die Übersetzung wird direkt nach dem DOM-Update erneut angestoßen.

Body-Hiding bei internen Seitenwechseln

PROBLEM

Eine frühere Lösung hat die komplette Seite auch bei internen SPA-Wechseln kurz versteckt. Das fühlte sich wie ein Ladefehler oder leerer Bildschirm an.

LÖSUNG

Body-Hiding wird nur noch beim initialen Laden verwendet. Bei internen Navigationen bleibt die Seite sichtbar und das Script aktualisiert den neuen DOM direkt.

Erste Übersetzung lief zu spät

PROBLEM

Die erste Übersetzung nach einem Route-Wechsel war anfangs an eine feste Verzögerung von 250 ms gebunden. Dadurch konnten Nutzer kurz unübersetzte Inhalte sehen.

LÖSUNG

Statt einer festen Wartezeit wird requestAnimationFrame genutzt. Die Übersetzung läuft dadurch im nächsten Render-Zyklus und damit deutlich früher.

Lazy-loaded Inhalte kamen später in den DOM

PROBLEM

Einige Inhalte wurden nicht sofort beim Seitenwechsel eingefügt, sondern zeitversetzt nachgeladen. Ein einzelner Übersetzungslauf konnte diese Inhalte verpassen.

LÖSUNG

Nach dem direkten Lauf werden zusätzliche Follow-up-Runs nach 100 ms, 400 ms und 1000 ms ausgeführt. Dadurch werden auch später gerenderte Inhalte verarbeitet.

3. DOM-Mutations und UI-Wechsel

Neue UI-Bereiche waren kurz unübersetzt

PROBLEM

Beim Wechsel innerhalb einer Seite, zum Beispiel von Aktiv auf Pausiert, wurden neue DOM-Nodes eingefügt und waren kurz auf Englisch sichtbar.

LÖSUNG

Neue DOM-Nodes werden im MutationObserver sofort verarbeitet. Der Debounce dient nur noch als Follow-up für nachgeladene Inhalte.

Text änderte sich ohne neue Elemente

PROBLEM

React kann bestehende Text-Nodes aktualisieren, ohne neue Elemente einzufügen. Solche Updates wurden von einer reinen childList-Beobachtung nicht zuverlässig erkannt.

LÖSUNG

Im MutationObserver wurde characterData: true aktiviert. Dadurch werden geänderte Text-Nodes direkt erkannt und erneut verarbeitet.

3. DOM-Mutations und UI-Wechsel

Attribute blieben kurz unübersetzt

PROBLEM

Texte in Attributen wie placeholder, title, aria-label und alt konnten bei Mutations kurz unübersetzt bleiben.

LÖSUNG

Attribute werden bei relevanten Mutations direkt über `processAttributes()` verarbeitet. Dadurch werden sichtbare und assistive UI-Texte konsistenter aktualisiert.

Debounce war für sichtbare UI-Wechsel zu langsam

PROBLEM

Ein Debounce von 200 ms war für schnelle UI-Wechsel zu träge und konnte sichtbare englische Zwischenzustände verursachen.

LÖSUNG

Der Debounce wurde auf 50 ms reduziert und dient nicht mehr als primärer Trigger für sichtbare Änderungen, sondern als Sicherheitslauf für später eintreffende Inhalte.

Split-Text bei Abrechnungsintervallen

PROBLEM

Der Satz *Billed every 4 weeks* wurde von StayAI in mehrere Text-Nodes zerlegt. Eine normale Regex-Übersetzung konnte den vollständigen Satz deshalb nicht zuverlässig erkennen.

LÖSUNG

Eine Spezialbehandlung für Split-Text-Nodes erkennt den Parent-Text, ändert aber nur die statischen Text-Nodes. Aus *Billed every* wird *Alle*, und der *Wochen-Text* wird zu *Wochen* abgerechnet.

Dynamische Zahlen-Nodes dürfen nicht zerstört werden

PROBLEM

Eine erste Split-Text-Lösung ersetzte den kompletten Satz durch einen einzelnen Text. Dadurch wurde der dynamische Zahlen-Node zerstört und beim Wechsel von 4 auf 8 Wochen blieb teilweise der alte Wert bestehen.

LÖSUNG

Die originale Node-Struktur bleibt erhalten. Der Zahlen-Node bleibt React-kontrolliert und kann weiterhin aktualisiert werden, während nur die statischen Textbestandteile übersetzt werden.

Kernaussage

Die Lösung ist bewusst DOM-schonend aufgebaut: dynamische React-kontrollierte Nodes bleiben erhalten, Formatierungen sind mehrfach ausführbar, und neue beziehungsweise geänderte Inhalte werden direkt über MutationObserver, Route-Hooks und gezielte Follow-up-Runs verarbeitet.